ChimeraBoost: Gradient Boosted Oblivious Trees in Pure Python

Version 0.33.0 · September 2026 · Apache-2.0 · github.com/bbstats/chimeraboost

Abstract

Gradient boosted decision trees remain the standard method for prediction on tabular data, and the leading implementations are large C++ systems. This paper describes ChimeraBoost, a gradient boosting library implemented entirely in Python on four dependencies (numpy, numba, scipy, and scikit-learn), with no compiled extension and no GPU requirement. On TabArena, a community benchmark run by its own maintainers and treated here strictly as a held-out evaluation, ChimeraBoost's default configuration ranks second among gradient boosting libraries, behind CatBoost and well ahead of XGBoost and LightGBM, while training in under half of CatBoost's median time. Much of the speed comes from a structure-replay refit: after early stopping, the selected tree structures are replayed against gradients computed on all rows, recovering the withheld validation data at negligible cost and reducing total fit time by 34.8% on a 59-dataset suite. Beyond the point prediction that TabArena measures, the library also estimates full predictive distributions: a gradient-matrix-free multi-quantile head fits a grid of conditional quantiles in one booster, with predictions that provably never cross, at approximately the per-round cost of a single level. Exact TreeSHAP attributions, conformalized prediction intervals, and temperature-calibrated probabilities are provided without external packages.

1. Introduction

The dominant gradient boosting implementations, XGBoost (Chen and Guestrin, 2016), LightGBM (Ke et al., 2017), and CatBoost (Prokhorenkova et al., 2018), owe their speed to substantial C++ codebases. That design choice carries a cost for research: the distance between an algorithmic idea and a testable implementation is large, and the population of practitioners who can modify these systems is small. ChimeraBoost explores the opposite corner of the design space. The entire library is Python, roughly 11,700 lines across 13 modules, with numba supplying JIT-compiled kernels where the profile demands them. The question the project asks is whether a library built this way can be competitive with the compiled systems rather than merely convenient, and the evidence presented below suggests that it can.

The contributions are as follows:

1. **A competitive pure-Python GBDT.** On the TabArena leaderboard the default configuration scores Elo 1297, second among gradient boosting libraries after CatBoost (1357) and ahead of

XGBoost (1208) and LightGBM (1181), at a median training time of 2.65 seconds per thousand rows against CatBoost's 5.88 (section 5).

2. **Structure-replay refit.** Early stopping withholds a validation fold, so the selected model has seen only part of the data. ChimeraBoost replays the selected tree structures against gradients computed on all rows, refitting leaf values only. This removed a step that had consumed 37 to 49% of every default fit and reduced total fit time by 34.8% on the decision suite, with accuracy unchanged (section 3).
3. **A gradient-matrix-free multi-quantile head.** A single booster estimates a grid of conditional quantiles (19 levels by default) with one tree structure per round and a vector-valued leaf. A projection collapses each row's K-channel pinball gradient to a scalar in one pass, so the (n × K) gradient matrix is never materialized and the head costs approximately what a single level costs per round. Predictions are monotone across levels by construction (section 4).
4. **An evaluation protocol with strict suite separation.** Development, tuning, published evidence, and external validation use disjoint dataset collections, and the external leaderboard is never consulted when making changes (section 5).

2. System overview

ChimeraBoost exposes a scikit-learn-compatible API of seven public names, three of which are estimators: `ChimeraBoostRegressor`, `ChimeraBoostClassifier`, and `ChimeraBoostQuantileRegressor`. Supported objectives include squared error, absolute error, Huber, Poisson, Gamma, Tweedie, and user-defined objectives for regression; binary and multiclass log loss for classification; and joint multi-quantile estimation.

Data handling. Categorical features are processed natively with ordered target statistics in the manner of CatBoost, averaged over four permutations, and may be addressed by column index or by DataFrame column name. Missing values receive a dedicated histogram bin at both fit and predict time, for numeric and categorical features alike. No one-hot encoding, imputation, or label encoding is required, and pandas is deliberately absent from the dependency list: DataFrames are consumed through their own `to_numpy` method.

Automatic configuration. Several decisions usually delegated to the user are made during the fit. Linear leaf models (Shi, Li and Li, 2019) are auditioned against constant leaves, and automatically generated cross features (Zhang et al., 2023) against plain features, in short races on a shared round budget; only the winner proceeds to full early stopping. The learning rate adapts to training set size, following the observation that dataset size is the one input CatBoost's own defaults respond to. Early stopping is enabled by default, and the stopped model is refit on all rows by structure replay. The remaining speed-accuracy choice is a single integer, `quality`, whose five settings were each measured to sit on the speed-accuracy frontier:

`quality`	profile	fit time (default = 1x)
1	fast	0.4x
2	balanced	0.8x
3	accurate (default)	1x
4	ensemble of 5	2.6x
5	ensemble of 8	3.8x

Interpretability and calibration. The `shap_values` method computes exact interventional TreeSHAP attributions (Lundberg et al., 2020) with no external package; the computation is exact over its background set, which defaults to at most 200 rows, and costs roughly 3 ms per row at depth 6. Classifier probabilities are temperature-scaled on the early-stopping fold; the transformation is monotone, leaving accuracy and AUC unchanged while improving calibration.

3. Training pipeline design

Oblivious trees. Every node at a given depth shares one (feature, threshold) pair, so a tree of depth d is described by d splits and a leaf index is a d-bit number (Kohavi and Li, 1995; Prokhorenkova et al., 2018). Prediction reduces to a handful of comparisons and a table lookup per tree, executed in a single numba kernel parallelized over samples so that each row traverses the entire forest while its features remain in cache. Sharing one split across a depth also acts as a strong regularizer.

Quantized histogram split finding. Splits are searched over binned features (128 bins by default), with gradients and hessians quantized to 15-bit integers following Shi, Ke et al. (2022). Integer histograms reduced fit time by 20 to 25% at unchanged accuracy in our measurements. Leaf values are always computed from exact floating-point gradients, and results are deterministic for a fixed seed. When row subsampling is enabled, rows are drawn by minimum-variance sampling (Ibragimov and Gusev, 2019), which weights selection by gradient magnitude.

Vector leaves. Multiclass models grow one tree per round rather than one per class, storing a class vector in each leaf (Iosipoi and Vakhrushev, 2022; Zhang and Jung, 2021). This made multiclass fits 2.5 times faster while improving Brier score by 5% at equal F1. The same mechanism supplies the quantile head's per-leaf vector.

Structure-replay refit. A model selected by early stopping has trained without the validation fold, and refitting from scratch on all rows had cost 37 to 49% of every default fit. Since growing trees accounts for 83 to 85% of fit time, ChimeraBoost instead replays the selected structure, applying the recorded splits round by round to gradients computed on the full data and refitting only the leaf values. On the 59-dataset decision suite this reduced total fit time by 34.8%, with 58 of 59 datasets fitting faster and accuracy statistically unchanged on four independent suites.

Bagging without replacement. The `n_ensembles` option fits members on 80% subsamples drawn without replacement, which outperformed the classical bootstrap on both accuracy and fit time in our comparisons. Members fit in parallel worker processes sharing one thread budget. With `refit_members=True`, each member is replayed on its full subsample; five refit members outperformed eight plain members on both accuracy and speed.

4. The multi-quantile head

`ChimeraBoostQuantileRegressor` estimates a grid of conditional quantiles jointly. From one fitted grid the model answers several distributional queries at no additional cost: central intervals, the median, the mean, the CDF at arbitrary thresholds, inverse-transform samples, and per-row exceedance probabilities via `predict_thresh(X, t)`.

Split search. Scoring K pinball objectives per candidate split would multiply the histogram work by K. Instead, a projection collapses each row's K-channel gradient to a scalar in a single pass, and split gains computed on the projected gradient land within 7% of the exact summed gain. The choice of projection matters: a naive sum across a symmetric grid is blind to dispersion, because the gradient contributions of levels tau and 1 - tau cancel exactly for an interval that is correctly centered but too narrow.

Non-crossing guarantee. Each row's predicted quantiles are sorted on output. Rearrangement never increases pinball loss at any level (Chernozhukov, Fernández-Val and Galichon, 2010), so the guarantee is free. The property is absent from per-level ensembles in practice: across 36 real regression datasets, LightGBM per-level boosters reversed 22% of adjacent level pairs on average and crossed on every dataset, and CatBoost's shared MultiQuantile head also crossed on every dataset. ChimeraBoost's crossing rate was exactly zero on all 36.

Calibration. The raw grid's nominal 80% and 90% intervals delivered 77% and 87% observed coverage, closer to nominal than LightGBM per-level (72% and 83%) or CatBoost MultiQuantile (73% and 83%). With `conformalize=True` the intervals gain distribution-free marginal coverage by conformalized quantile regression (Romano, Patterson and Candès, 2019), measured within 2.7 percentage points of nominal at n = 10,000. An out-of-fold study on seven datasets confirmed the derived quantities: exceedance probabilities showed expected calibration error from 0.029 to 0.051, crossing remained zero everywhere, and the raw grid's slight narrowness was confined to the outermost tail cells. Conformalization corrected precisely that, bringing 90% coverage to between 0.902 and 0.905 on the three worst datasets, at a cost of about one point of CRPS skill.

Attribution of interval width. Because both edges of an interval come from one model, SHAP attributions can be computed for the width of an interval, identifying the features that make a given row's prediction uncertain rather than high or low. A stack of independently fitted per-level models cannot produce this decomposition.

Comparison. On the 36 datasets, with every method early-stopping on the same split: against 19 independent LightGBM quantile boosters, CRPS was approximately tied (23 wins, 13 losses), while the interval score, a proper rule pricing coverage and width jointly, favored ChimeraBoost 31 to 5 at two thirds of the fit time. CatBoost's MultiQuantile won on CRPS; section 6 quantifies this.

5. Evaluation

Protocol. Benchmark suites are assigned single, non-overlapping roles: a synthetic screen for mechanism checks; a decision suite (Grinsztajn et al., 2022, plus a frozen high-cardinality collection) on which changes are accepted or rejected by per-stratum sign tests; a tuning suite (PMLB), the only data hyperparameters are fitted to; a 22-dataset independently audited public suite for published evidence; and TabArena, which is reported and never consulted during development. A suite used for tuning cannot also measure overfitting to itself, and datasets enter a suite on data properties alone (row counts, cardinality, missingness), never on observed results. When early speed claims for the quantile head (3 to 6 times LightGBM) were traced to fixed-round fits on synthetic data, the documentation was corrected to the real-data figure of 1.5x, with the correction noted in place.

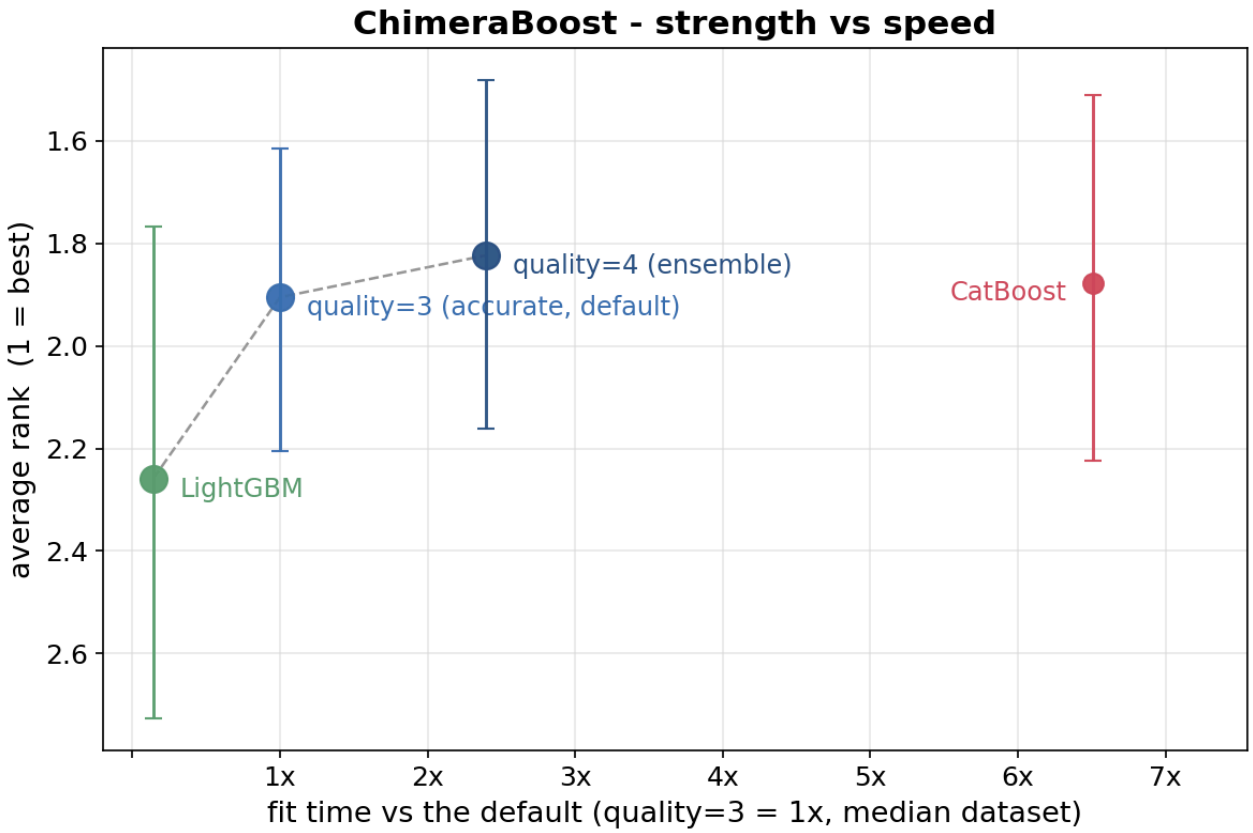
TabArena. TabArena (Erickson et al., 2025) is a leaderboard of 51 curated classification and regression datasets in which every model is run by the benchmark's maintainers and ranked by Elo. As of September 2026, in the leaderboard's default view, gradient boosting libraries at default settings ranked as follows:

model (default config)	Elo	median train s/1K	median predict s/1K
CatBoost	1357	5.88	0.025
ChimeraBoost	**1297**	**2.65**	**0.052**
XGBoost	1208	1.94	0.123
EBM	1191	6.67	0.014
LightGBM	1181	1.96	0.142

ChimeraBoost's default placed second, with the 89-point gap to third place exceeding the 60-point gap to first. It trained in 45% of CatBoost's median time per thousand rows and predicted two to three times faster than XGBoost and LightGBM defaults, though CatBoost's predict time remains lower. The tuned-and-ensembled entry scored 1360, marginally above CatBoost's default, using a search space that predates several of the library's current adaptive defaults. The top of the overall board is held by tabular foundation models near Elo 1750, pretrained networks in a different resource class. The evaluated entry ran version 0.30.0; the two releases since left default-configuration predictions unchanged bit for bit, so the accuracy figures carry over, and later speedups mean the timing column, if anything, understates the current version. Since no TabArena result has ever influenced a change to the library, these figures constitute an out-of-sample evaluation.

Public suite. The published chart runs on 22 audited public datasets disjoint from all development data. In the most recent full run (August 2026, three seeds, 21 of 22 datasets scored), with fit times expressed relative to the ChimeraBoost default:

model	avg rank	95% CI	median fit time (default = 1x)
ChimeraBoost `quality=4` (ensemble)	**1.82**	1.48-2.16	2.4x
CatBoost	1.88	1.51-2.22	6.5x
ChimeraBoost default	1.90	1.62-2.21	1.0x
LightGBM	2.26	1.77-2.73	0.1x



Each ChimeraBoost configuration is ranked in its own three-way contest against CatBoost and LightGBM, so 2.0 is the middle of the field. Ranks are facet-balanced across task type, dataset size, and cardinality; the unweighted ranks, which favor CatBoost slightly, are recorded in the repository. The default configuration was statistically indistinguishable from CatBoost at about a sixth of CatBoost's fit time on the median dataset, and the `quality=4` ensemble achieved the best average rank at just over a third of CatBoost's cost.

6. Limitations

CRPS against CatBoost MultiQuantile. CatBoost's shared quantile head won 29 of 36 datasets on CRPS, a real deficit. It required a median 8.3 times ChimeraBoost's fit time and was substantially worse calibrated (worst coverage error 0.64 against a nominal 0.90, versus 0.10 for ChimeraBoost), so the interval score favored ChimeraBoost 25 to 11; applications that weight CRPS alone and disregard cost are better served by CatBoost.

Compiler cold start. The first fit after a fresh install or upgrade incurs roughly ten seconds of numba compilation (0.8 s warm, 0.27 s in steady state). A `chimeraboost-warmup` command and an environment variable allow precompilation; the cost is reduced rather than eliminated.

Training speed, especially at scale. LightGBM and XGBoost defaults train in roughly 75% of ChimeraBoost's time at TabArena's scale, and the gap grows with row count: in internal measurements on synthetic data at a fixed tree count, the fit-time ratio to LightGBM widened from about 5x at 50,000 rows to roughly 10x at 500,000, because numba's generated histogram kernels do not match hand-tuned C++ SIMD. The absence of GPU support, a consequence of the pure-Python design, bites in exactly this regime: the compiled libraries all offer GPU training whose payoff arrives on datasets of millions of rows, and no pure-Python counterpart exists. The fit-time comparisons in this paper describe the small-to-medium regime the benchmarks cover and should not be extrapolated to very large data. Prediction throughput is less affected; at two million rows it measured on par with LightGBM. The library's case rests on quality per unit of compute rather than minimum training time.

API stability. The library is at version 0.x; names have been stable in practice but are not yet guaranteed.

7. Availability

ChimeraBoost is available on PyPI (`pip install chimeraboost`, Python 3.9+) under the Apache-2.0 license.

```
from chimeraboost import ChimeraBoostClassifier, ChimeraBoostQuantileRegressor

clf = ChimeraBoostClassifier()                    # or quality=4 for the ensemble
clf.fit(X_train, y_train, cat_features=["city"])  # categoricals by name, NaNs fine
proba = clf.predict_proba(X_test)                 # temperature-calibrated

qr = ChimeraBoostQuantileRegressor(conformalize=True)
qr.fit(X_train, y_num, cat_features=["city"])
lo, hi = qr.predict(X_test, kind="interval", alpha=0.1).T   # guaranteed 90% band
p_over = qr.predict_thresh(X_test, 100.0)                   # P(y > 100) per row
```

Documentation is at bbstats.github.io/chimeraboost. The benchmark harness, raw results, and the studies cited here are in the repository under `benchmarks/`: the TabArena figures are the leaderboard read of 2026-09-01 (`benchmarks/tabarena/LEADERBOARD_SNAPSHOT.md`), the public-suite run is recorded in `benchmarks/PUBLIC_PLAN.md`, the quantile comparisons come from `benchmarks/quantile_suite.py` (full tables), and the out-of-fold calibration study is `benchmarks/quantile_oof_calibration.py`. The large-data scaling measurements come from `benchmarks/scaling_giant.py` and `benchmarks/scaling_predict.py`. Per-feature attributions for borrowed techniques are maintained on the attribution page.

References

- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*.
- Chernozhukov, V., Fernández-Val, I. and Galichon, A. (2010). Quantile and probability curves without crossing. *Econometrica*.
- Erickson, N. et al. (2025). TabArena: A living benchmark for machine learning on tabular data. *NeurIPS*.
- Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS*.
- Ibragimov, B. and Gusev, G. (2019). Minimal variance sampling in stochastic gradient boosting. *NeurIPS*.
- Iosipoi, L. and Vakhrushev, A. (2022). SketchBoost: Fast gradient boosted decision tree for multioutput problems. *NeurIPS*.
- Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS*.
- Kohavi, R. and Li, C.-H. (1995). Oblivious decision trees, graphs, and top-down pruning. *IJCAI*.
- Lundberg, S. et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *NeurIPS*.
- Romano, Y., Patterson, E. and Candès, E. (2019). Conformalized quantile regression. *NeurIPS*.
- Shi, Y., Ke, G., Chen, Z., Zheng, S. and Liu, T.-Y. (2022). Quantized training of gradient boosting decision trees. *NeurIPS*.
- Shi, Y., Li, J. and Li, Z. (2019). Gradient boosting with piece-wise linear regression trees. *IJCAI*.
- Zhang, T. et al. (2023). OpenFE: Automated feature generation with expert-level performance. *ICML*.
- Zhang, Z. and Jung, C. (2021). GBDT-MO: Gradient-boosted decision trees for multiple outputs. *IEEE Transactions on Neural Networks and Learning Systems*.